

Set 1 - OpenMP, MPI, and Python

Issued: November 26, 2025

Question 1: Parallel Numerical Optimization

Εισαγωγή

Η ολική βελτιστοποίηση αποτελεί ένα από τα σημαντικότερα θέματα έρευνας στον τομέα των εφαρμοσμένων μαθηματικών. Έχει πολύ σημαντικές εφαρμογές σε διάφορα πεδία επιστημών, όπως οικονομία, χημεία, βιολογία και μηχανική.

Στόχος της ολικής βελτιστοποίησης είναι η εύρεση μίας λύσης εντός ενός συνόλου λύσεων στο οποίο η αντικειμενική συνάρτηση αποκτά τη μικρότερη τιμή της, δηλαδή το ολικό ελάχιστο. Επομένως, η ολική βελτιστοποίηση δεν στοχεύει απλά στον καθορισμό ενός τοπικού ελαχίστου αλλά στο μικρότερο τοπικό ελάχιστο από το σύνολο των λύσεων.

Για την εύρεση ενός τοπικού ελαχίστου έχουν προταθεί διάφορες μέθοδοι. Ο αλγόριθμος MDS (Multi-Dimensional Search) λειτουργεί σε ένα πολύτοπο (simplex) σημείων του χώρου αναζήτησης. Ο αλγόριθμος εφαρμόζει ένα σύνολο από πράξεις (reflections, expansions, contractions) στο simplex ώστε να εξασφαλίσει τη σύγκλιση σε ένα τοπικό ελάχιστο. Ο αλγόριθμος είναι ντετερμινιστικός, δεν βασίζεται σε τυχαίους αριθμούς και δεν χρησιμοποιεί παραγώγους, παρά μόνο κλήσεις της συνάρτησης που μπορούν να εκτελεστούν ταυτόχρονα.

Σε κάθε επανάληψη t του αλγορίθμου, διατηρείται ένα simplex διάστασης n :

$$S(t) = \{x_0^{(t)}, x_1^{(t)}, \dots, x_n^{(t)}\}, \quad x_i^{(t)} \in \mathbb{R}^n.$$

Το κέντρο του simplex αποτελεί μια προσέγγιση στο ελάχιστο. Η τιμή της συνάρτησης υπολογίζεται στις n κορυφές του simplex και η καλύτερη κορυφή ορίζεται ως αυτή με τη μικρότερη τιμή συνάρτησης. Οι κορυφές αναδιατάσσονται έτσι ώστε η μετάβαση στην επόμενη επανάληψη να πραγματοποιείται περιστρέφοντας το simplex γύρω από το σημείο $x_0^{(t)}$ και επιχειρώντας μία ανάκλαση. Στη συνέχεια, η αντικειμενική συνάρτηση αξιολογείται στις n κορυφές του ανακλώμενου simplex. Εάν η τιμή της συνάρτησης μειωθεί, τότε επιχειρείται ένα βήμα επέκτασης για την παραγωγή ενός ακόμη μεγαλύτερου ανακλώμενου simplex. Εάν η ανάκλαση δεν παράγει μικρότερη τιμή συνάρτησης (από αυτή του άξονα περιστροφής), τότε επιχειρείται ένα βήμα συστολής που μειώνει το μέγεθος του simplex.

Η διαδικασία επαναλαμβάνεται μέχρι να ικανοποιηθεί ένα κριτήριο τερματισμού. Αυτό το κριτήριο μπορεί να είναι ο μέγιστος αριθμός επαναλήψεων, ο μέγιστος αριθμός κλήσεων της συνάρτησης ή ο εντοπισμός μη περαιτέρω προόδου σε μία επανάληψη.

Σας δίνεται μια υλοποίηση της μεθόδου MDS στο αρχείο `torczon.c`. Η συνάρτηση υπολογίζει ένα σημείο x στο οποίο η μη γραμμική συνάρτηση $f(x)$ έχει ένα τοπικό ελάχιστο. Το σημείο x είναι ένα n -διάστατο διάνυσμα και η τιμή της συνάρτησης $f(x)$ είναι βαθμωτή, δηλαδή ισχύει:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}.$$

Η αντικειμενική συνάρτηση που θα χρησιμοποιηθεί στα πλαίσια της εργασίας του μαθήματος είναι η συνάρτηση Rosenbrock, η οποία ορίζεται ως εξής:

$$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2].$$

Η συνάρτηση έχει καθολικό ελάχιστο στο σημείο $(1, 1, \dots, 1)$, όπου παίρνει την τιμή 0. Για περισσότερες πληροφορίες: http://en.wikipedia.org/wiki/Rosenbrock_function.

Μια προφανής πιθανοτική διαδικασία ολικής βελτιστοποίησης βασίζεται στην εφαρμογή ενός αλγορίθμου τοπικής βελτιστοποίησης σε διαφορετικά σημεία της συνολικής περιοχής αναζήτησης. Η διαδικασία καλείται *Multistart* (Πολυεναρκτήριο Τοπική Αναζήτηση), με τα σημεία στα οποία εφαρμόζεται η τοπική βελτιστοποίηση να επιλέγονται με τυχαίο τρόπο.

Λογισμικό Ολικής Βελτιστοποίησης

Η παρούσα εργασία περιλαμβάνει μία εφαρμογή (`multistart_mds_seq.c`) που υλοποιεί σειριακά τη μέθοδο Multistart σύμφωνα με την περιγραφή στην προηγούμενη ενότητα. Η εφαρμογή επιλέγει `ntrials` τυχαία σημεία διάστασης `nvars`, καλεί τη μέθοδο MDS σε καθένα από αυτά για να βρει το τοπικό ελάχιστο και υπολογίζει το σημείο εκείνο που δίνει το ολικό ελάχιστο για τη συνάρτηση Rosenbrock, εκτυπώνοντας τελικά το σημείο αυτό καθώς και την τιμή της συνάρτησης στο συγκεκριμένο σημείο. Ως πεδίο αναζήτησης χρησιμοποιείται το διάστημα $[-2, 2)$ για κάθε μία από τις `nvars` διαστάσεις της συνάρτησης.

Παράλληλη Ολική Βελτιστοποίηση

Βασικός στόχος της εργασίας είναι η παραλληλοποίηση του λογισμικού ολικής βελτιστοποίησης, χρησιμοποιώντας τα διάφορα μοντέλα προγραμματισμού που μελετήθηκαν στα πλαίσια του μαθήματος. Συγκεκριμένα, η μέθοδος MDS θα πρέπει να εφαρμοστεί παράλληλα στα τυχαία σημεία που επιλέγονται από την εφαρμογή, ελαχιστοποιώντας έτσι τον χρόνο εύρεσης του σημείου που αντιστοιχεί στο ολικό ελάχιστο.

Τα μοντέλα προγραμματισμού τα οποία θα χρησιμοποιήσετε για την παραλληλοποίηση της εφαρμογής είναι τα ακόλουθα:

- **OpenMP**: πολλαπλά νήματα εφαρμόζουν τη μέθοδο MDS στα τυχαία σημεία.
- **OpenMP tasks**: όπως πριν, αλλά χρησιμοποιώντας το μοντέλο εργασιών του OpenMP. Στην περίπτωση αυτή καλείστε να αξιοποιήσετε και τον παραλληλισμό εντός της μεθόδου MDS.
- **MPI**: πολλαπλές διεργασίες εφαρμόζουν τη μέθοδο στα τυχαία σημεία.

Μετρήσεις

Αφού ολοκληρώσετε τις παράλληλες εκδόσεις της εφαρμογής και βεβαιωθείτε για τη σωστή λειτουργία τους, θα μετρήσετε την απόδοσή τους ώστε να δείξετε τη σωστή αξιοποίηση του διαθέσιμου πολυεπεξεργαστικού υλικού. Για κάθε μοντέλο προγραμματισμού θα πρέπει να μετρήσετε τον χρόνο εκτέλεσης της εφαρμογής, για διάφορους αριθμούς επεξεργαστικών στοιχείων (νημάτων / διεργασιών κλπ). Για κάθε μοντέλο, μας ενδιαφέρει τόσο ο ελάχιστος χρόνος εκτέλεσης όσο και η χρονοβελτίωση (speedup) της εφαρμογής.

Ως σημείο αναφοράς για τα τελικά σας πειράματα, χρησιμοποιήστε τη συνάρτηση Rosenbrock για αρχικά 64 σημεία διάστασης 4.

Question 2: Parallel Parametric Search in Machine Learning

Εισαγωγή

Η τυχαία παραμετρική αναζήτηση (random search) αποτελεί μία αποδοτική μέθοδο για τη βελτιστοποίηση υπερπαραμέτρων, ειδικά όταν ο χώρος τιμών είναι μεγάλος και η εξαντλητική αναζήτηση πλέγματος δεν είναι πρακτική. Στην παρούσα άσκηση εξετάζουμε την τυχαία αναζήτηση αποκλειστικά ως προς μία μόνο υπερπαραμέτρο: την παράμετρο κανονικοποίησης C του ταξινομητή λογιστικής παλινδρόμησης (Logistic Regression).

Το σύνολο δεδομένων δημιουργείται συνθετικά με:

- μεγάλο πλήθος χαρακτηριστικών ($n_features = 200$),
- πολύ μικρή πραγματική πληροφορία ($n_informative = 10$),
- συνολικά 100000 δείγματα,
- μέτρια διαχωρισιμότητα ($class_sep = 1.2$),
- εισαγωγή θορύβου στις ετικέτες ($flip_y = 0.08$),

ώστε να προσομοιώνεται ένα υπολογιστικά απαιτητικό, υψηλής διάστασης πρόβλημα. Πριν την εκπαίδευση εφαρμόζεται κανονικοποίηση χαρακτηριστικών με StandardScaler, ώστε η τιμή του C να έχει πιο συνεπή επίδραση στη διαδικασία βελτιστοποίησης.

Η τυχαία αναζήτηση υλοποιείται δειγματοληπτώντας τιμές του C από λογαριθμικά ομοιόμορφη κατανομή στο διάστημα:

$$C \sim 10^{U[-5, -1]},$$

όπου U είναι η ομοιόμορφη κατανομή. Για κάθε τιμή του C , εκπαιδεύεται ένα μοντέλο με ποινή ℓ_1 ($penalty = "l1"$) και $solver = "saga"$, κατάλληλο για μεγάλα, υψηλής διάστασης προβλήματα.

Ο σειριακός κώδικας μοιάζει ως εξής:

```

1 import numpy as np
2 from sklearn.datasets import make_classification
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.metrics import accuracy_score
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 X, y = make_classification(n_samples=100000, n_features=200, n_informative=10,
9                             n_redundant=10, class_sep=1.2, flip_y=0.08, random_state=42)
10
11 X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, stratify=y,
12                                                    random_state=42)
13
14 scaler = StandardScaler()
15 X_train = scaler.fit_transform(X_train)
16 X_val = scaler.transform(X_val)
17
18 rng = np.random.default_rng(0)
19 results = []
20
21 for i in range(32):
22     C = 10 ** rng.uniform(-5, -1)
23     clf = LogisticRegression(
24         C=C, penalty="l1", solver="saga",
25         max_iter=3000, n_jobs=1
26     )
27     clf.fit(X_train, y_train)
28     acc = accuracy_score(y_val, clf.predict(X_val))
29     print(f"{i:02d}: C={C:.3e}, acc={acc:.4f}")
30     results.append((C, acc))
31
32 best_C, best_acc = max(results, key=lambda t: t[1])
33 print(f"Best: C={best_C:.3e}, acc={best_acc:.4f}")

```

Παράλληλη Τυχαία Αναζήτηση

Η εκτέλεση κάθε δοκιμής είναι ανεξάρτητη, επομένως η διαδικασία είναι ιδανική για παραλληλοποίηση επιπέδου εργασιών. Η παράλληλη υλοποίηση θα γίνει με:

- concurrent futures,
- mpi4py futures,
- υλοποίηση με MPI (mpi4py).

Μετρήσεις

Αφού ολοκληρώσετε τις παράλληλες εκδόσεις:

- μετρήστε τον χρόνο εκτέλεσης για διαφορετικό αριθμό διεργασιών/νημάτων,
- υπολογίστε τη χρονοβελτίωση (speedup) έναντι της σειριακής εκτέλεσης,
- σχολιάστε την κλιμάκωση για μικρό και μεγάλο αριθμό δοκιμών.

Μπορείτε να αυξήσετε τον αριθμό δοκιμών (`n_trials`) ώστε ο χρόνος να είναι επαρκής για αξιόπιστη αξιολόγηση της απόδοσης.

Παραδοτέα

Κώδικας (σε αρχείο zip) και αναφορά (σε αρχείο pdf) με ανάλυση του προβλήματος, εξήγηση της λύσης, και παρουσίαση των αποτελεσμάτων.